

Handoff & Repo Structure

Exact folder structure, file ownership, and how to send work to the Team Lead without using GitHub.

Read this if: everyone — read this alongside Team Workflow

Since only the Team Lead (you) uses GitHub directly, everyone else works locally on a fixed set of files and hands them off as zips. This document is the exact structure, naming, and handoff process — follow it precisely and integration stays predictable.

Repository skeleton

The Team Lead creates every file below in the repo before anyone starts coding — even empty. Nobody creates new top-level files inside another member's folder without telling the Team Lead first.

```
code-archaeologist/
├── backend/app/
│   ├── main.py [M1]
│   ├── core/config.py, security.py [M1]
│   ├── api/routes_*.py (7 files) [M1]
│   ├── schemas/*.py (7 files) [M1]
│   ├── models/db_models.py [M1]
│   ├── services/integration/pipeline.py [M1]
│   ├── services/repository/ [M2]
│   │   ├── cloner.py, validator.py, file_tree.py, language_detector.py
│   ├── services/history/ [M2]
│   │   ├── commits.py, pull_requests.py, issues.py, contributors.py, timeline.py
│   ├── services/parsing/ [M3]
│   │   ├── python_parser.py, javascript_parser.py, entity_extractor.py, dependency_mapper.py
│   ├── services/retrieval/ [M3]
│   │   ├── embeddings.py, vector_search.py
│   ├── services/knowledge/ [M3]
│   │   ├── knowledge_store.py, gap_detector.py
│   ├── services/llm/ [M3]
│   │   ├── groq_client.py, prompt_templates.py
│   ├── services/risk/ [M3]
│   │   ├── risk_scorer.py, signals.py
│   ├── agents/ [M3]
│   │   ├── historian.py, guidance_agent.py, knowledge_gap_agent.py
│   ├── agents/offboarding/ [M3]
│   │   ├── question_generator.py, report_builder.py
│   ├── graph/ [M3]
│   │   ├── workflow.py, nodes.py, state.py
├── frontend/src/
│   ├── pages/ [M4]
│   │   ├── Dashboard.tsx, RepositoryDetail.tsx, Login.tsx
│   ├── components/layout/ [M4]
│   │   ├── Navbar.tsx, Sidebar.tsx
│   ├── features/auth/ [M4]
│   │   ├── LoginForm.tsx, useAuth.ts
│   ├── features/repositories/ [M4]
│   │   ├── RepositoryList.tsx, RepositoryImportForm.tsx, AnalysisProgress.tsx
│   ├── features/explorer/ [M4]
│   │   ├── FileTree.tsx, CodeViewer.tsx
│   ├── services/apiClient.ts [M4]
│   ├── types/index.ts [M4]
│   ├── features/architecture/ [M5]
│   │   ├── DependencyGraph.tsx
│   ├── features/history/Timeline.tsx [M5]
│   ├── features/historian/ [M5]
│   │   ├── QuestionBox.tsx, AnswerCard.tsx, EvidencePanel.tsx
│   ├── features/guidance/ [M5]
│   │   ├── GuidanceForm.tsx, StepList.tsx
│   ├── features/risk/RiskDashboard.tsx [M5]
│   ├── features/knowledge/ [M5]
│   │   ├── KnowledgeGapList.tsx, AnswerForm.tsx
│   ├── features/offboarding/ [M5]
│   │   ├── OffboardingWorkspace.tsx, ReportView.tsx
├── .env.example
├── .gitignore
└── README.md
```

Golden rule

Edit only the files marked with your member number. If two members touch the same file, whoever didn't own it caused the conflict.

How to send your work to the Team Lead

1. **Zip only your own folder(s)** — not the whole project. Example: M2 zips `services/repository/` and `services/history/` together.

2. **Name the zip exactly:** M<number>_<ShortModuleName>_<YYYYMMDD>.zip
Examples: M2_RepositoryHistory_20260916.zip , M3_ParsingAndAI_20260916.zip , M4_FrontendShell_20260916.zip , M5_FrontendIntelligence_20260916.zip
3. **Upload to one shared folder** (e.g. a Google Drive folder named Code_Archaeologist_Handoffs). Don't send via scattered DMs or overwrite someone else's file there.
4. **Before zipping, check:** it runs locally, no secrets are in it, and you haven't touched a file outside your own folder.

What the Team Lead does on receiving a zip

1. Download and unzip directly into the matching path — paths match 1:1, so it's a straight drop-in
2. Diff against the previous version as a quick sanity check
3. Commit with a clear message, e.g. M2: adds commit/PR history extraction – 2026-09-16
4. Push to develop , run integration checks, merge to main once stable

Keep this moving both ways. Members can't git pull , so they have no way to see what's already been integrated. After every integration — especially one that changes a shared schema or type — export the current state of the relevant folders and send it back as their new starting point. As a rule, push updates out to the team at least as often as they send work in.

Who reads what

Everyone reads **1. Project Overview, 7. Team Workflow, 8. Contributing Guidelines**, and this document first. Beyond that:

Member	Must read	Can skip
M1 — Team Lead	All 9 documents — you own integration, so every contract, schema, and ownership line matters to you.	Nothing
M2 — Repository & Git History	2. Product Requirements · 3. Technical Build Spec · 4. Team Structure & Ownership · 6. Database Design (the tables you write to: repositories, commits, pull_requests, issues, contributors)	5. API Contract (M1 wraps your output — you don't call the API yourself)
M3 — Parsing / AI / Risk / Offboarding	2. Product Requirements · 3. Technical Build Spec · 4. Team Structure & Ownership · 5. API Contract (you power these endpoints) · 6. Database Design (in full — you touch the most tables)	Nothing — this role needs the most context
M4 — Frontend A: Shell & Workspace	2. Product Requirements · 4. Team Structure & Ownership · 5. API Contract (repositories, files, analysis endpoints especially)	6. Database Design · 3. Technical Build Spec (skim scope section only)
M5 — Frontend B: Intelligence & Knowledge	2. Product Requirements · 4. Team Structure & Ownership · 5. API Contract (questions, guidance, risk, knowledge, offboarding endpoints especially)	6. Database Design · 3. Technical Build Spec (skim scope section only)

If you only have time for one thing before you start coding: read the "Must read" column for your own row, plus the repo skeleton at the top of this document.